16831 Course Project Proposal

Mind the Touch Gap: Real-to-Sim Contact Alignment for Dexterous Pen Spinning

Xinyao Li
xinyaol2
xinyaol2@andrew.cmu.edu

Wen-Han Hsieh
wenhanh
wenhanh@andrew.cmu.edu

Seungjae Baek
seungjab
seungjab@andrew.cmu.edu

Shuai Zhou
shuaizho
shuaizho@andrew.cmu.edu

Yuan Li
yuanli5
yuanli5@andrew.cmu.edu

Yusu Fang
yusuf
yusuf@andrew.cmu.edu

Abstract: Contact-rich, dynamic in-hand skills such as pen spinning remain difficult to transfer from simulation. PenSpin [1] improved its hardware performance by fine-tuning a proprioceptive student on a small set of successful open-loop replays. We ask whether the same limited target experience can instead improve the contact observations used during simulator training. We propose to replay source actions in a target simulator, learn a mapping from source link-level contact to target contact, freeze that mapping in the source simulator, and re-distill a contact-aware student. Isaac Gym [2] supplies source training; Project SuperDex [3] supplies a controlled target with observable contacts. We compare against the original proprio-only student, raw-contact training, randomized contact, simple calibration, and PenSpin-style target fine-tuning. This sim-to-sim proxy tests the value of contact-observation alignment; it does not claim to correct contact dynamics or establish hardware transfer.

Keywords: Dexterous manipulation, contact alignment, sim-to-sim transfer

Team size: 6.

1 Introduction

Simulation-based reinforcement learning has enabled in-hand manipulation skills that are hard to teleoperate [4, 5, 6]. Pen spinning is a demanding case: it is fast, contact-rich, and unforgiving of small timing errors. PenSpin [1] trained a privileged oracle and distilled a student using proprioception alone. Its sim-trained student did not transfer reliably to hardware, even after substantial simulation work.

PenSpin's adaptation replayed oracle actions open-loop on hardware, retained successful trajectories, and fine-tuned the student. This uses little target data, but it adapts a particular policy rather than improving the contact observations available in the training simulator. We investigate whether replay data can be used at the observation level, so a policy can continue learning from target-like touch cues in the source simulator.

Prior work has adapted simulator parameters from target experience [7, 8, 9], and recent work has calibrated tactile simulation through replay [10]. Our proposed intervention is narrower: it aligns a policy's contact input while leaving the source collision solver, friction, and object motion unchanged. Because PenSpin's original student has no contact input, we explicitly add a contact-aware student and retain the original proprio-only student as a baseline.

We evaluate this idea in a sim-to-sim setting, with Isaac Gym as the source and SuperDex [3] as the target. The target simulator permits contact labels and controlled resets that are difficult to obtain on hardware. We can vary friction and compliance, measure state divergence during paired replays, and evaluate every policy on matched target initial states. The setting is a controlled proxy, not a real-world result.

We ask three questions:

RQ1: Under changes in target friction and compliance, when does a contact input help or hurt transfer relative to proprioception alone?

RQ2: How much target performance does a learned contact observation mapping recover from a small replay dataset?

RQ3: How does observation alignment compare with target-policy fine-tuning at the same number of replay attempts?

2 Method

We retain PenSpin's privileged oracle, reproduce its proprio-only student, and add a student with binary link-level contact. The proposed third stage aligns that contact input and re-distills the contact-aware student (Figure 1). We call Isaac Gym the source domain $\mathcal{S}$ and SuperDex the target domain $\mathcal{T}$.

Setup. An Allegro hand spins pen-like objects using PenSpin's released task, action space, and PD control. The common observation $c_t \in \{0,1\}^K$ indicates which of K corresponding hand links touch the pen. Source contact forces and pooled target readings are thresholded at the same policy observation rate. Link mapping and thresholds are fixed using training and validation replays, never target test episodes. We aim to match object geometry, action timing, initial-state distribution, and rotation/drop criteria across simulators.

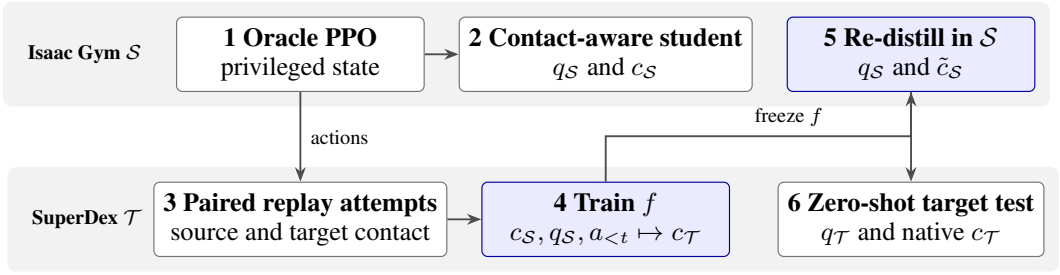


Figure 1: Six-stage pipeline. Paired replay attempts train the contact observation mapper f; source-domain rollouts train the student. The target simulator supplies native contact for evaluation, while target-policy fine-tuning is a separate baseline.

1. **Oracle.** Train a privileged PPO oracle in $\mathcal{S}$ with object state and physical parameters, following PenSpin's reward and control setup.
2. **Student.** Reproduce the proprio-only student and distill a separate contact-aware student from the same oracle. The latter observes a history of proprioception q and binary contact $c_\mathcal{S}$.
3. **Replay and collect.** From matched initial states, replay the same oracle actions open-loop in $\mathcal{S}$ and $\mathcal{T}$. Log contact, joint and object state, and preceding actions in both domains. All attempted replays supply contact labels, including failures; the fine-tuning baseline may filter successes.
4. **Contact alignment.** Train a small temporal mapper to predict target binary contacts from source observations available before action a_t:

$$p_{\mathcal{T},t} = f_\phi(c_{\mathcal{S},t-H:t}, q_{\mathcal{S},t-H:t}, a_{t-H:t-1}), \quad \mathcal{L} = \mathrm{BCE}(p_{\mathcal{T},t}, c_{\mathcal{T},t}). \tag{1}$$

Choose a per-link threshold τ on validation replays and form $\tilde{c}_{\mathcal{S},t} = \mathbf{1}[p_{\mathcal{T},t} > \tau]$.

5. **Re-distill.** Freeze f and distill the contact-aware student again from the fixed oracle in $\mathcal{S}$, supplying $\tilde{c}_{\mathcal{S}}$ in the same binary format as target contact.
6. **Deploy.** Evaluate the re-distilled student in $\mathcal{T}$ with native binary target contact. The mapper is not run in $\mathcal{T}$.

Time-indexed open-loop replays can drift into different physical states. We will measure joint- and object-state divergence and report contact prediction accuracy by divergence bin. In a separate diagnostic, we reset $\mathcal{T}$ to source states before matched steps to create state-matched contact pairs. This requires privileged simulator access and is neither a deployable hardware-data baseline nor a mathematical upper bound.

3 Experiments

All policies use the same target test objects, initial states, and evaluation episodes. Table 1 retains the baseline comparison from our original proposal. Methods using target data receive the same number of *attempted* replay episodes; for PenSpin-style behavior cloning, we additionally report how many were successful.

Method	Target data used for	Role
Proprio-only student	—	Original PenSpin input baseline
Raw-contact student	—	Contact without alignment
Contact randomization (noise, dropout, delay)	—	Robustness baseline
Parametric contact calibration [10]	Threshold and delay fitting	Simple sensor correction
PenSpin-style target fine-tuning [1]	BC on successful replays	Policy adaptation
Learned contact alignment	Training f	Proposed method
State-matched contact pairs	Training f with resets	Diagnostic with extra access

Table 1: Methods compared under matched target test conditions. Target-policy fine-tuning uses target training data and is evaluated on held-out target episodes; it is not zero-shot.

Metrics. Policy metrics are full rotations, time until drop, and success rate at a fixed rotation criterion. Sensor metrics on held-out replay attempts are per-link contact precision, recall, F1, and onset timing error. We will report mean and variability across evaluation seeds together with replay-state divergence.

Data efficiency (RQ2, RQ3). We compare 10, 25, 50, and 100 attempted target replays. Calibration and alignment can use labels from all attempts. PenSpin-style fine-tuning filters successful trajectories as in the original method, so we will report its successful demonstration count at each equal-attempt budget. Training, validation, and contact test sets are split by entire replay, not by time step.

Diagnostic comparisons (RQ1). We compare each student's source and target performance, then vary target friction and compliance. Proprio-only transfer, raw-contact transfer, and aligned-contact transfer show which interventions help. Observation errors and contact dynamics interact, so their performance differences are not additive estimates of separate gap components. The state-matched control tests whether time-indexed pairing error limits the mapper.

4 Plan

The largest engineering risk is porting the Allegro/pen environment to SuperDex. We begin with a short hand-control/contact replay smoke test before scaling data collection.

- **10/09–10/23:** Reproduce the PenSpin oracle and proprio-only student; implement and validate the shared link-level contact representation.
- **10/24–11/06:** Port the Allegro hand, pen, and PD controller to SuperDex; match control timing and collect paired replay attempts.
- **11/07–11/20:** Train the contact mapper, re-distill the contact-aware student, and obtain first zero-shot target results.
- **11/21–12/04:** Run equal-attempt baselines, data-budget sweeps, friction/compliance variations, and state-matched diagnostics.
- **12/05–12/11:** Prepare the final report and presentation.

Fallbacks. If full pen spinning is unstable in the port, we will first test a simpler in-hand rotation task [4]. If the SuperDex port stalls, we will use a target simulator with explicit link-level contact labels and state the change. If the target gap is too small, we will widen it through controlled friction and compliance changes. These outcomes would remain simulation evidence rather than hardware transfer.

Acknowledgments

ChatGPT assisted with editorial revision and LaTeX formatting.

References

[1] J. Wang, Y. Yuan, H. Che, H. Qi, Y. Ma, J. Malik, and X. Wang. Lessons from learning to spin "pens". In *Conference on Robot Learning (CoRL)*, 2024. URL https://penspin.github.io/.

[2] V. Makoviychuk et al. Isaac gym: High performance GPU-based physics simulation for robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[3] The Mochi Team. Project SuperDex: A unified platform for dexterous manipulation research. Open-source software, Meta (facebookresearch), 2026. URL https://github.com/facebookresearch/project_superdex.

[4] H. Qi, A. Kumar, R. Calandra, Y. Ma, and J. Malik. In-hand object rotation via rapid motor adaptation. In *Conference on Robot Learning (CoRL)*, 2022.

[5] OpenAI et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.

[6] H. Qi, B. Yi, S. Suresh, M. Lambeta, Y. Ma, R. Calandra, and J. Malik. General in-hand object rotation with vision and touch. In *Conference on Robot Learning (CoRL)*, 2023.

[7] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox. Closing the sim-to-real loop: Adapting simulation randomization with real world experience. In *International Conference on Robotics and Automation (ICRA)*, 2019.

[8] T. He et al. ASAP: Aligning simulation and real-world physics for learning agile humanoid whole-body skills. In *Robotics: Science and Systems (RSS)*, 2025.

[9] F. Ramos, R. Possas, and D. Fox. BayesSim: Adaptive domain randomization via probabilistic inference for robotics simulators. In *Robotics: Science and Systems (RSS)*, 2019.

[10] S. Luo, X. Huang, Z. Xu, W. Li, Z. Jiao, and C. Xiao. Blind dexterous grasping via Real2Sim2Real tactile policy learning, 2026. URL https://arxiv.org/abs/2606.11767.